

How SQL Expressions Can Be Combined

A practical roadmap for learning how CASE, aggregate functions, scalar functions, arithmetic, subqueries, CTEs, and window functions can be combined.

1. The Core Idea

SQL expressions can be used as inputs to functions, and functions can be nested inside other functions. Learn to read complex SQL from the inside outward.

General pattern:

```
FUNCTION(EXPRESSION)
```

Nested pattern:

```
OUTER_FUNCTION(INNER_FUNCTION(EXPRESSION))
```

2. CASE as an Expression

Basic CASE

```
CASE WHEN marks >= 50 THEN 'Pass' ELSE 'Fail' END
```

CASE returns a value

```
CASE WHEN gender = 'M' THEN marks ELSE NULL END
```

Multiple conditions

```
CASE WHEN marks >= 90 THEN 'A' WHEN marks >= 75 THEN 'B' ELSE 'C' END
```

3. Expressions Inside Aggregate Functions

SUM + CASE

```
SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)
```

AVG + CASE

```
AVG(CASE WHEN gender = 'M' THEN marks END)
```

MIN + CASE

```
MIN(CASE WHEN gender = 'M' THEN marks END)
```

MAX + CASE

```
MAX(CASE WHEN gender = 'M' THEN marks END)
```

COUNT + CASE

```
COUNT(CASE WHEN gender = 'M' THEN 1 END)
```

4. CASE + GROUP BY + Aggregates

Class-wise male average

```
SELECT class,  
AVG(CASE WHEN gender = 'M' THEN marks END) AS male_avg  
FROM students  
GROUP BY class;
```

Class-wise boys/girls

```
SELECT class,
SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS boys,
SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS girls
FROM students
GROUP BY class;
```

5. Functions Inside Functions

ROUND + AVG

```
ROUND(AVG(marks), 2)
```

COALESCE + AVG

```
COALESCE(AVG(marks), 0)
```

ROUND + AVG + CASE

```
ROUND(AVG(CASE WHEN gender = 'M' THEN marks END), 2)
```

UPPER + CASE

```
UPPER(CASE WHEN gender = 'M' THEN 'Male' ELSE 'Female' END)
```

6. Arithmetic + Functions

Average calculated manually

```
SUM(marks) / COUNT(*)
```

Percentage

```
SUM(marks) * 100.0 / SUM(max_marks)
```

Adjusted marks

```
marks + CASE WHEN marks < 50 THEN 10 ELSE 0 END
```

7. Aggregate Functions Inside CASE

Class rating

```
CASE
WHEN AVG(marks) >= 80 THEN 'Excellent'
WHEN AVG(marks) >= 60 THEN 'Good'
ELSE 'Needs Improvement'
END
```

Pass/fail based on class average

```
CASE
WHEN AVG(marks) >= 50 THEN 'Class Passed'
ELSE 'Class Needs Improvement'
END
```

8. Conditional Aggregation

Count boys

```
SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)
```

Total marks for boys

```
SUM(CASE WHEN gender = 'M' THEN marks ELSE 0 END)
```

Average marks for boys

```
AVG(CASE WHEN gender = 'M' THEN marks END)
```

Highest marks among boys

```
MAX(CASE WHEN gender = 'M' THEN marks END)
```

9. CASE + Multiple Conditions

Boys who passed

```
COUNT(CASE  
  WHEN gender = 'M' AND marks >= 50 THEN 1  
  END)
```

High-scoring girls

```
SUM(CASE  
  WHEN gender = 'F' AND marks >= 80 THEN 1  
  ELSE 0  
  END)
```

10. Subqueries as Expressions

Above overall average

```
SELECT student_name, marks  
FROM students  
WHERE marks > (SELECT AVG(marks) FROM students);
```

CASE using a subquery

```
CASE  
  WHEN marks > (SELECT AVG(marks) FROM students)  
  THEN 'Above Average'  
  ELSE 'Not Above Average'  
  END
```

11. CTE + Aggregation + CASE

Class performance

```
WITH class_avg AS (  
  SELECT class, AVG(marks) AS avg_marks  
  FROM students  
  GROUP BY class  
)  
SELECT class,  
  avg_marks,  
  CASE WHEN avg_marks >= 80 THEN 'Excellent'  
  WHEN avg_marks >= 60 THEN 'Good'  
  ELSE 'Needs Improvement' END AS rating  
FROM class_avg;
```

12. Window Functions + Expressions

Class average without collapsing rows

```
AVG(marks) OVER (PARTITION BY class)
```

Difference from class average

```
marks - AVG(marks) OVER (PARTITION BY class)
```

Conditional window aggregation

```
SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)
OVER (PARTITION BY class)
```

Rank with CASE

```
CASE WHEN marks >= 80 THEN
RANK() OVER (PARTITION BY class ORDER BY marks DESC)
END
```

13. A Powerful Reading Method

Read inside → outside

```
ROUND(AVG(CASE WHEN gender = 'M' THEN marks END), 2)
```

Step 1

```
CASE WHEN gender = 'M' THEN marks END → male marks or NULL
```

Step 2

```
AVG(...) → average of male marks
```

Step 3

```
ROUND(..., 2) → round the average to 2 decimals
```

14. Recommended Learning Order

Stage 1

```
SELECT → WHERE → ORDER BY → expressions
```

Stage 2

```
COUNT/SUM/AVG/MIN/MAX → GROUP BY → HAVING
```

Stage 3

```
CASE WHEN → NULL → COALESCE
```

Stage 4

```
CASE inside aggregates → aggregates inside CASE → nested functions
```

Stage 5

```
JOINS → subqueries → CTEs
```

Stage 6

```
Window functions → CASE + windows → aggregate + windows
```

Stage 7

```
PIVOT/ROLLUP/GROUPING SETS → optimization → advanced SQL
```

Quick Reference

Combination	What it does
SUM(CASE...)	Conditional total/count
AVG(CASE...)	Average only matching rows
COUNT(CASE...)	Conditional count
MIN(CASE...)	Conditional minimum
MAX(CASE...)	Conditional maximum
ROUND(AVG(...),2)	Round an aggregate result
CASE WHEN AVG(...)...	Make a decision from an aggregate
CASE + GROUP BY	Class/group-level conditional analysis
CASE + OVER()	Conditional window calculation

Key principle: CASE produces a value for each row; aggregate functions combine those values. Once this becomes natural, complex SQL becomes much easier to read and construct.